

FusedLinearAttention

FusedLinearAttention is an HPML course project on collapsing Q, K, and V projection plus scaled dot-product attention into a single CUDA kernel. The repo now contains:

- a complete ETTh1 + PatchTST baseline pipeline
- a canonical CUDA extension path under `kernel/`
- correctness, profiling, validation, and figure-generation workflows
- final H100 experiment artifacts under `baseline_pipeline/results`

The short version: the fused kernel is **correct** and it **reduces estimated HBM traffic**, but on the current implementation it is still **slower than the PyTorch baseline** on H100 because the kernel is still dominated by scalar fp32 projection work.

Project Goal

The unfused attention path does this in two stages:

1. project input tokens into `Q`, `K`, and `V`
2. read those intermediates back to run attention

The fused kernel aims to:

- reduce kernel launches
- reduce global-memory traffic
- avoid materializing full `Q/K/V` tensors in HBM

Final Status

What is complete

- ETTh1 preprocessing and DataLoaders
- PatchTST baseline model, training, and evaluation
- baseline profiling workflow
- canonical NumPy / PyTorch correctness oracle
- canonical CUDA kernel source and PyTorch extension binding

- fused profiling pipeline
- Phase 3 end-to-end validation pipeline
- H100 compiled-kernel correctness run
- H100 fused-vs-baseline profiling run

What is still not ideal

- the fused kernel does **not** beat the unfused PyTorch baseline on H100
- end-to-end fused model quality is worse than baseline
- backward is still a custom autograd bridge, not a handwritten fused CUDA backward kernel

Repository Layout

baseline_pipeline/

- ETTh1 preprocessing
- PatchTST baseline
- baseline and fused profiling scripts
- Phase 3 end-to-end validation
- result merge and figure generation

kernel/

- `fused_attn.cu`
- `fused_attn_ext.cpp`
- `load_kernel.py`
- kernel design notes

tests/

- correctness checks
- reference comparisons
- fused backward smoke test

CPU_Reference_in_NumPy/

- original NumPy oracle
- golden outputs

- reference comparison utilities

Final H100 Results

Source files:

- comparison_table.csv
- validation_table.csv
- endtoend_timing.csv
- correctness_results.csv

Forward microbenchmark

Seq Len	Baseline (us)	Fused (us)	Fused vs Baseline
64	109.7448	872.0335	0.1258x
128	109.1533	1422.1433	0.0768x
256	108.9229	2532.7407	0.0430x
512	118.5030	4737.3379	0.0250x
1024	198.5365	9124.8203	0.0218x

Forecast metrics

Model	MSE	MAE
Baseline	180.46336365	12.64666843
Fused	213.44288635	13.50915241

Delta vs baseline:

- MSE: +18.274913%
- MAE: +6.81985129%

End-to-end timing

Model	Mean Epoch Time (s)	Forward / iter (ms)
Baseline	1.8825	0.544459
Fused	2.5510	0.633303

Generated Figures

The main charts are already in `baseline_pipeline/results/figures`:

- `speedup.png`
- `wall_time_speedup.png`
- `hbm_bandwidth.png`
- `kernel_count.png`
- `occupancy_vs_tile.png`
- `occupancy_vs_tile_tradeoff.png`
- `nsight_timeline_comparison.png`

There is also a detailed written report here:

- `baseline_pipeline/final_report.md`

How To Run

1. Baseline pipeline

```
cd baseline_pipeline
python model/data.py
python profiling/baseline_bench.py
python model/train.py
python model/evaluate.py
```

2. Correctness

Quick CUDA check:

```
python tests/test_correctness.py --quick
```

Full CUDA check:

```
python tests/test_correctness.py
```

Simulation/reference mode:

```
python tests/test_correctness.py --simulate --quick --use-root-oracle  
python tests/test_fused_backward.py
```

3. Fused profiling

```
cd baseline_pipeline  
./run_bhanuja.sh
```

4. Phase 3 validation

```
cd baseline_pipeline  
./run_phase3.sh --train-fused
```

5. Full helper

```
./run_all.sh
```

Environment Notes

For compiled-kernel runs you need:

- CUDA toolkit with `nvcc`
- a CUDA-compatible PyTorch build
- `ninja`
- a GPU where `torch.cuda.is_available()` is true

The H100 run used:

- host CUDA toolkit
- H100 GPU
- `TORCH_CUDA_ARCH_LIST=9.0`
- compiled extension path via `kernel/load_kernel.py`

Why The Fused Kernel Is Still Slower

Even after the successful optimizations, the current fused kernel still loses to the PyTorch baseline because:

- Q/K/V projection is still implemented as scalar fp32 loops inside the kernel
- the baseline uses highly optimized library kernels
- reduced HBM traffic is being outweighed by compute inefficiency

The kernel is therefore:

- **correct**
- **more memory-efficient on paper**
- **not yet compute-efficient enough to win on H100**

Best Current Optimization State

The best version in the repo right now includes:

- block-per-query-tile launch structure
- dynamic shared memory
- projection tiling through shared memory
- architecture-aware compilation
- a small launch-bounds hint

That version is much faster than the older fused kernel revisions that were committed earlier in the branch, even though it still trails the baseline.

Recommended Next Work

If this project is extended further, the highest-return next steps are:

1. replace scalar projection loops with tensor-core-friendly MMA tiling
2. move toward fp16/bf16 compute paths
3. redesign work partitioning at the warp level for projection and score accumulation
4. implement a true fused CUDA backward kernel

Team Contributions

Jnanasree Konda

- fused-attention NumPy oracle and correctness work
- PyTorch extension interface
- kernel validation and GPU experiment integration

Bhanuja Karumuru

- fused kernel design and CUDA implementation
- profiling pipeline, result merge, and figure generation

Rithwik Amajala

- ETTh1 pipeline
- PatchTST baseline, training, and evaluation
- Phase 3 model-side integration and validation workflow